

Modul 6 Sekuensial dan Konkuren

Tujuan Praktikum

1. Praktikan memahami perbedaan konsep sekuensial dan konkuren pada Xinu
2. Praktikan mampu membuat program konkuren pada Xinu

6.1 Pendahuluan

1. Secara intuisi, ketika program berjalan, programmer membayangkan komputer akan melakukan eksekusi kode baris per baris, statement per statement. **Pada setiap waktu komputer hanya mengeksekusi sebuah statement.** Cara pandang tersebut disebut sebagai sekuensial.
2. Sistem operasi memperluas pandangan mengenai komputasi dengan konsep *concurrent processing*. *Concurrent processing* berarti banyak komputasi dapat berlangsung “pada saat yang bersamaan”. Catatan: pada komputer dengan banyak core atau cpu, komputer benar-benar dapat melakukan banyak hal secara bersamaan (*parallelism*). Pada komputer dengan 1 CPU maka OS akan menciptakan ilusi konkurensi.
3. Untuk menciptakan ilusi “berjalan pada saat yang bersamaan”, komputer menggunakan teknik yang disebut multiprogramming/multitasking/interleaving yaitu sistem operasi memilih proses yang ada dari kumpulan proses yang tersedia, mengeksekusi proses tersebut, memilih proses lain dan kemudian mengkesekusinya. Komputer akan mengkesekusi satu proses dalam beberapa millisecond kemudian berpindah ke proses lainnya. Ketika dilihat oleh manusia, tampak bahwa semua proses berjalan bersamaan karena cepatnya prosesor dan lambatya indera manusia.
4. Terdapat 2 kategori multitasking yaitu: timesharing dan realtime. Timesharing memberikan prioritas yang sama untuk setiap komputasi, memperbolehkan komputasi berjalan dan berakhir sesuai dengan durasi yang telah ditentukan saja. Realtime memberikan prioritas yang berbeda untuk setiap komputasi karena terdapat performansi waktu yang ingin dicapai. Realtime akan memberikan prioritas dan menjadwalkan proses secara terencana.

6.2 Sekuensial Program

Pada **sekuensial program**, prosesor akan mengeksekusi instruksi satu per satu. Perhatikan kode main.c berikut ini:

```

1  /* main.c - main */
2
3  #include <xinu.h>
4  void sndA(void);
5  void sndB(void);
6
7  process main(void)
8  {
9
10     sndA();
11     sndB();
12     return OK;
13 }
14
15 void sndA(void)
16 {
17
18     while(1) {
19         printf("A\n");
20         sleepms(1000);
21     }
22 }
23
24 void sndB(void)
25 {
26
27
28     while(1) {
29         printf("B\n");
30         sleepms(1000);
31     }
32 }
33

```

Gambar 6-1 Contoh Sekuensial Program pada Xinu

Pada kode tersebut terdapat **sebuah proses** yaitu proses main (baris 7) dan **dua buah fungsi** yaitu sndA() dan sndB(). Ketika program dieksekusi, proses main akan memanggil 2 buah fungsi yaitu sndA() dan sndB(). Jika sudah selesai memanggil dua fungsi tersebut maka proses main akan terminasi. Mohon pahami perbedaan konsep proses dan fungsi!

Fungsi sndA() akan menampilkan huruf A (baris 19) kemudian sleep selama 1 detik (baris 20) menggunakan system call sleepms() yang disediakan oleh Xinu. Fungsi sndA() akan terus menerus berjalan karena ada while(1). Jadi output fungsi sndA(): print "A", sleep selama 1 detik, print "A", sleep 1 detik, dst. Fungsi sndB() sama dengan sndA() hanya menampilkan huruf B secara terus menerus.

Fungsi akan berjalan secara sekuensial. Sedangkan proses akan berjalan secara konkuren. Konsekuensi dari kode di atas adalah: fungsi sndB() tidak akan pernah dieksekusi karena fungsi sndA()

tidak pernah terminasi (selalu berada pada loop while (1){}). Tidak akan muncul huruf “B”, hanya selalu muncul huruf “A”.

6.3 Konkuren Program

Berikut adalah contoh **konkuren** pada sistem operasi Xinu

```
1  /* main.c - main */
2
3  #include <xinu.h>
4  void sndA(void);
5  void sndB(void);
6
7
8  process main(void)
9  {
10     resume(create(sndA, 1024, 20, "process A", 0));
11     resume(create(sndB, 1024, 20, "process B", 0));
12     return OK;
13 }
14
15 void sndA(void)
16 {
17     while(1){
18         printf("A\n");
19         sleepms(1000);
20     }
21 }
22
23
24 void sndB(void)
25 {
26     while(1){
27         printf("B\n");
28         sleepms(1000);
29     }
30 }
31
32
33 }
```

Gambar 6-2 Contoh Konkuren Program pada Xinu

Perbedaan utama kode ini dengan kode sebelumnya adalah **terdapat 3 proses dalam kode ini yaitu proses main, proses A dan process B**. Pada kode sebelumnya hanya terdapat 1 proses yaitu proses main saja dan 2 fungsi. Untuk membuat proses baru dapat dicapai menggunakan kode baris ke 10 dan 11. Perhatikan baris 10, create() adalah syscall pada Xinu. Syscall create() akan membuat proses baru bernama “process A”. Output dari syscall create () adalah PID proses yang baru saja dibuat. Pada Xinu, proses yang baru saja dibuat berada dalam state suspend. Oleh karena itu, digunakan syscall resume() untuk menjalankan proses tersebut. Baris ke 11, sama dengan baris 10 hanya saja proses bernama “process B” dan menggunakan fungsi sndB().

Berikut adalah deklarasi syscall create():

```

11 pid32  create(
12      void      *funcaddr, /* Address of the function */
13      uint32     ssize,    /* Stack size in bytes */
14      pri16      priority, /* Process priority > 0 */
15      char       *name,    /* Name (for debugging) */
16      uint32     nargs,    /* Number of args that follow */
17      ...
18  )

```

Gambar 6-3 Deklarasi Syscall create()

create(nama_fungsi_dipanggil, ukuran_stack, prioritas, nama_proses, banyaknya_argument)

create(sndA, 1024, 20, "process A, 0) berarti:

- + fungsi yang akan digunakan dalam proses tersebut adalah sndA()
- + ukuran stack adalah 1024
- + proses tersebut mempunyai prioritas 20
- + nama proses tersebut adalah "process A"
- + tidak ada argument yang digunakan pada fungsi sndA() sehingga ditulis 0

Misalkan kita mempunyai fungsi sum(int a, int b) dan akan membuat proses tersebut maka
create(sum, 1024, 20, "penjumlahan", 2, int a, int b);

resume(pid_proses_yang_akan_diresume)

Setelah syscall resume() dieksekusi proses akan aktif dan dijadwalkan pada CPU. Hasil dari kode di atas adalah output A B muncul bergantian

